

Project Explained — eBPF-Based Granular Energy Profiler

A plain-language reference for what this project is, what Review 2 required, what was built, and how. Keep this alongside [README.md](#) (the technical run/setup instructions) and [docs/Review2_Progress_Document.docx](#) (the graded submission).

1. What Review 2 required

Per [BCSE307_Project_Review_2_Guidelines](#), Review 2 is **implementation-focused** — diagrams, literature survey and future plans alone are not sufficient. The team had to show:

- Working code for the major modules
(front-end/analysis/optimization/etc., as applicable to the project)
- An **integrated prototype** with meaningful end-to-end or module-to-module execution
- Repository evidence with member-wise commits and readable structure
- Testing evidence (valid, invalid, boundary, project-specific cases)
- A defect/risk log and a completion plan for Review 3
- Every member able to individually explain and demonstrate their assigned module

Graded across 7 components worth 10 marks total: corrections/progress (1.0), module implementation (2.5), integration/prototype (1.5), testing/debugging (1.5), code quality/repo (1.0), completion plan (0.5), demonstration/viva (2.0).

2. What the project is (from Review 1)

Team A27 — eBPF-Based Granular Energy Profiler for Green Cloud Computing.

Problem: Hardware energy counters (Intel RAPL) only report power at the

CPU-socket level. If ten microservices share one host, you can't tell how many Joules any single one of them burned — existing tools either poll `/proc` periodically (missing short-lived scheduling events) or are heavyweight ML-based systems built for full Kubernetes stacks (Kepler).

Solution: Use **eBPF** to trace the kernel's own scheduler (`sched_switch`) with nanosecond precision, tracking exactly how long each process spends on-CPU. Then take the RAPL socket-wide Joule reading for a time window and split it across processes in proportion to their share of that window's CPU time. Result: real-time, per-process energy attribution, without touching or slowing down the processes being measured.

The three modules the team designed at Review 1: 1. **Kernel Probe** — eBPF program tracking active CPU nanoseconds per PID. 2. **Hardware Reader** — daemon reading RAPL's `energy_uj` sysfs counter. 3. **Attribution Logic** — merges 1 and 2, distributes Joules per PID.

3. What was built for Review 2

All three modules, plus the integration layer that ties them together, all runnable and tested:

	File(s)	What it does
1	<code>sched_probe.c</code>	C eBPF program attached to <code>tracepoint:sched_switch</code> ; maintains a BPF

	File(s)	What it does
l P r o b e	py	hash map of cumulative on-CPU nanoseconds per PID. The Python loader polls that map and converts it into per-window deltas.
2 — H a r d w a r e	rapl_reader.py	Reads <code>/sys/class/powercap/intel-rapl:0/energy_uj</code> , converts successive readings into Joules-per-

Read the file(s) and return

Read the file(s) and return

3

—

AttributeError: 'int' object has no attribute 'readlines'

Enginn

File(s)
)

attrib
ution
.py

What it
does

window, correctly handling the counter's wraparound.

Pure function: given each PID's CPU-time and the window's total Joules, splits the Joules proportionally. No I/O, so it's fully unit-testable on its own.

	File(s)	What it does
e		
l		
n		Runs all
t		three
e		every
g		polling
r		window,
a	ener	prints a
ti	gy_p	live per-
i	rofile	process
o	r.py	table, and
n		writes a
		JSON
C		report at
L		the end.
I		

Plus: - **20 automated tests** ([tests/](#)) — all passing — covering the attribution math, RAPL wraparound handling, and the probe's windowing logic. - [docs/architecture.md](#) — full data-flow diagram and module interface table. - [docs/Review2_Progress_Document.docx](#) (+ PDF) — the actual document to submit, following the guideline's 14-section structure, with real evidence pulled from this repo (test counts, a defect log, a member-wise contribution table).

4. How it was built

eBPF and RAPL both require a real Linux kernel with root access — this dev machine doesn't have that. So every module was written with **two code paths**:

- `run_real()` — the actual production path: real BCC-loaded eBPF, real `sysfs` reads. Written correctly against the real APIs, but not yet exercised on hardware.
- `run_simulated()` (`--simulate` flag) — synthetic PIDs and wattage standing in for the kernel/hardware, so the *entire* pipeline (windowing → attribution → CLI reporting) can run and be verified end-to-end on any machine, right now.

The `--simulate` run shown during the build produced output like this (5 windows, deterministic seed):

```
=== Run Totals ===
```

PID	NAME	TOTAL JOULES
1003	pid-1003	77.0581
1001	pid-1001	59.9763
1004	pid-1004	48.2509
1002	pid-1002	40.3720

That confirms the integration actually works: scheduling data in, Joules

attributed out, numbers that sum back correctly (verified by the test suite too).

5. What's still open (see progress document §12 for full detail)

1. **Run the real eBPF/RAPL path on an actual Linux VM** — the top item before Review 3. The code is written for it; it just hasn't been executed against a live kernel yet.
2. **Idle-window energy** currently goes unattributed rather than being assigned to a modeled "idle" bucket — logged as a known limitation, not yet fixed.
3. **Cross-check against an external power meter / stress-ng** using [scripts/run_stress_test.sh](#), once on real hardware.
4. Fill in the **Review Date** and **Faculty name** placeholders in the progress document, and push the repo so each member's real commits appear.

6. Quick reference — how to see it work yourself

```
# run the whole pipeline (works on Windows/Mac/Linux, no root
needed)
```

```
python -m src.cli.energy_profiler --duration 10 --interval 1 --
simulate --seed 42
```

```
# run the test suite
```

```
python -m pytest tests/ -v
```